Day 5

MongoDB, Debugging & Production Practices

Three things that must land — everything else is negotiable.

By **Wesam Abousaid**



3 Things That Must Land

Everything else is negotiable

The Race Is Real

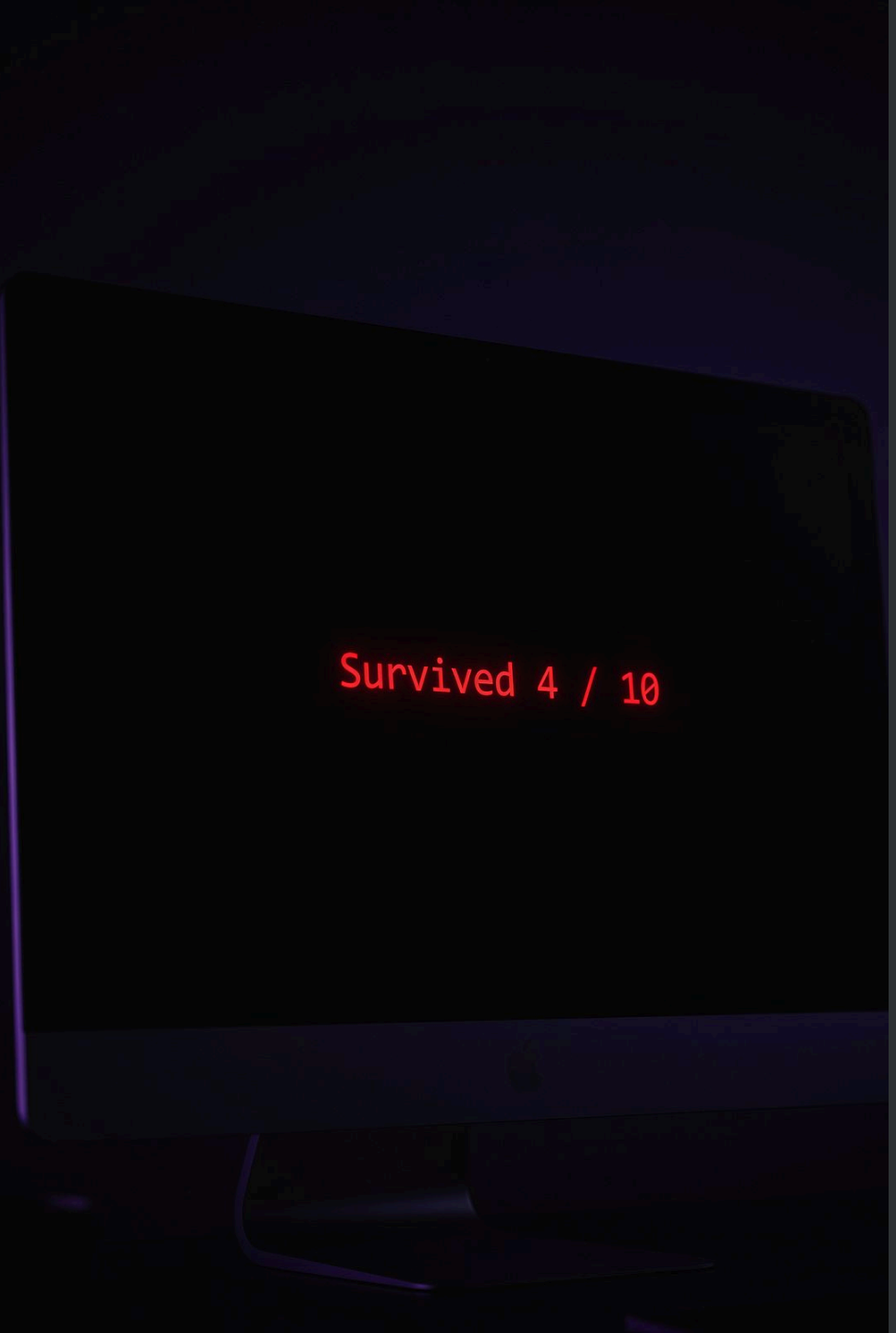
Measured, not asserted. Every student must personally witness the stress test failing — seeing `Survived: 4 / 10` on their own screen. Not described, not shown on a projector. Witnessed.

The Repository Interface Is the Actual Work

MongoDB is the easy part. The hard part — and the day's real lesson — is understanding why `readTasks` / `writeTasks` had to die, and what a proper storage interface looks like. Students should be able to articulate this.

The Swap Costs One `require`

The entire week's architectural claim, finally cashed. The stress test reads `10 / 10`. The `public/` directory was never opened. The frontend never found out. That is the proof.



DEMO → DO

Cold Open: Break It

1. **Run the Stress Test** — Run `node scripts/stress.js` on the projector against file storage. No preamble — just run it.
2. **Students Run It** — Reset `data/tasks.json` to `[]` first. Three runs. Write the numbers down. Every student runs it on their own machine.
3. **Debrief on the Evidence** — Walk through the interleaving timeline and the failure table. Point at the gap between `readTasks()` and `writeTasks()` — every `await` is a place Node goes and does someone else's work. That gap is the entire bug.
   The failure table: duplicate ids from `Date.now()`, no queries, O(file) writes, one machine forever.

> Every one of those requests returned 201 Created. Every one fired a `task:created` broadcast, and every open tab rendered it. Six of those tasks do not exist. Nothing in your logs will ever tell you that. This is a lost update — the most dangerous class of bug there is, because every individual step reported success.

TALK — LAPTOPS CLOSED

Why a Database

The Comparison Table

Map one-to-one onto the failures students just measured:

Challenge	File Storage	Database
Concurrent writes	Race conditions, lost updates	Atomic operations, locking
Querying	Load everything, filter in Node	Query at the source
Multiple servers	Impossible — one file, one machine	Shared, networked storage
Data size	O(file) reads and writes	Indexed, partial reads
Schema validation	None — anything goes	Enforced at write time

SQL ↔ Mongo Vocabulary

A lookup table:

SQL	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
JOIN	$lookup / embedding

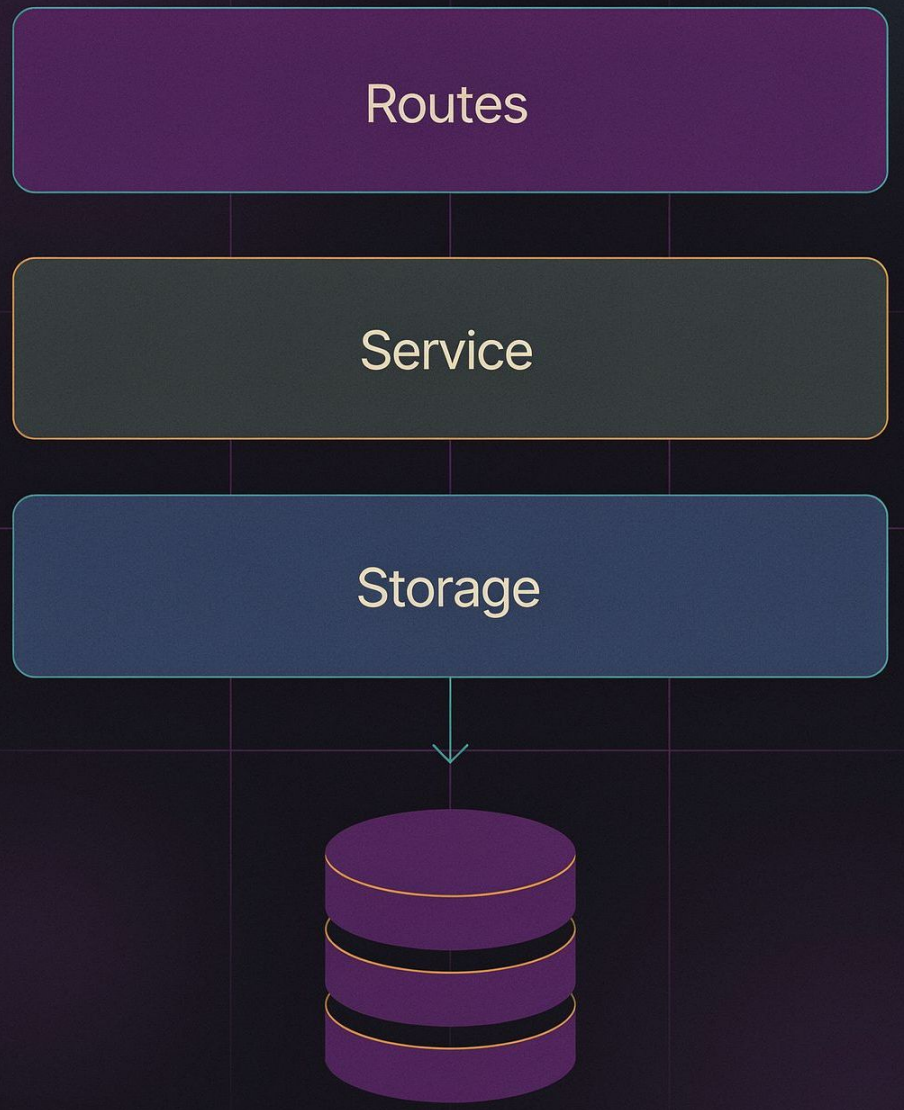
> A good architecture makes the hard things easy and makes the wrong things obvious.

The Honesty Box

Postgres is the better default for many systems. MongoDB is here because it maps onto the JS objects students already have — and the choice is swappable. Postgres would be a third file in `storage/`.

The Architectural Claim

The database is not the point. The repository interface is the point. The database is just the thing that slots into the interface — which is exactly what the rest of the session will prove. A Postgres implementation would be a third file in `storage/` and the service would never know.



CODE-ALONG

Connect Early: dotenv + db.js

Config Is Environment

Point at process.env.PORT || 3000 from Day 3: students have been doing this all week without noticing. Environment variables are not a new concept — they are a pattern they already use.

1. Create .env with MONGODB_URI set to the connection string
2. Create .env.example with the variable name but no value — this is what gets committed
3. Confirm .gitignore already has .env — it was added on Day 2. The plan was made in advance.

storage/db.js

Small file — type it, don't paste it. Two deliberate choices:

- No default URI fallback. A server that starts without its database is a server that answers every request with an error while looking perfectly healthy to whoever deployed it. Crash loudly at boot instead.
- The env var name is MONGODB_URI — pick one name, use it everywhere. A mismatch here will burn twenty minutes of class time.

server-express.js

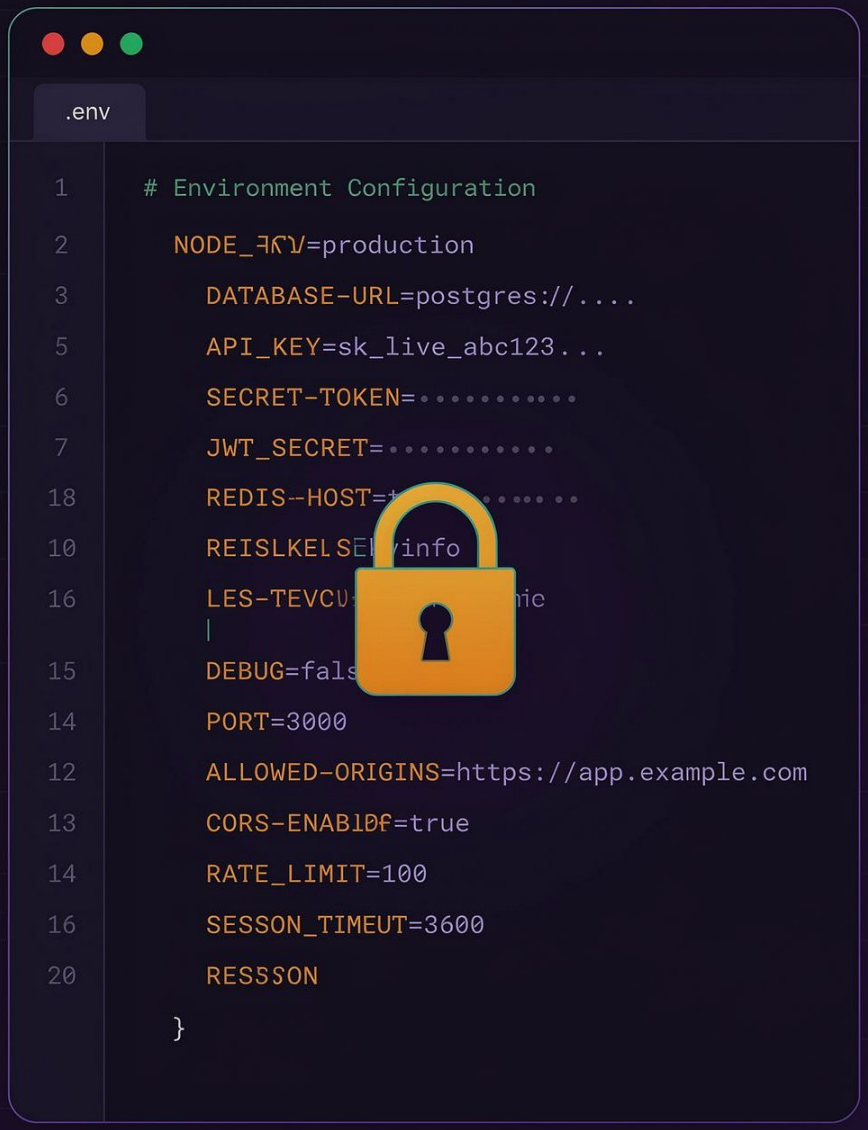
1. `require("dotenv").config()` as line 1 of the entry point — any module that reads process.env at require-time, required above that call, sees undefined. That bug takes an hour to find.
2. Wrap startup in an async start() function
3. `await connectDB()` before `app.listen()` — the connection must be established before the server accepts traffic

> dotenv.config() is line 1 of the entry point for a reason. Any module that reads process.env at require-time, required above that call, sees undefined. That bug takes an hour to find.

> A server that starts without its database is a server that answers every request with an error while looking perfectly healthy to whoever deployed it. Crash loudly at boot instead.

Sabotage Test

Break the password in .env, restart the server, and watch it exit with a clear error message. Then fix it. This is the fail-fast principle in action — and it is the moment students understand why the default URI fallback was deliberately omitted.



THE HEART OF THE DAY

★ The Repository Refactor

The Problem

Two functions on screen, alone:

```
readTasks()       // give me everything
writeTasks(tasks)  // replace everything
```

> This isn't a storage interface. It's a file, wearing a costume.

Scroll through Day 4's `taskService.js` and count the file mechanics — `find`, `filter`, `push`, `Date.now()`, `toISOString()`. None of it is business logic. It is all plumbing that belongs in storage.

The killer argument: implement `writeTasks` against MongoDB and you fetch and replace the entire collection on every request. The file's exact problems, on better hardware.

The Contract Rule

Storage reports. The service decides.

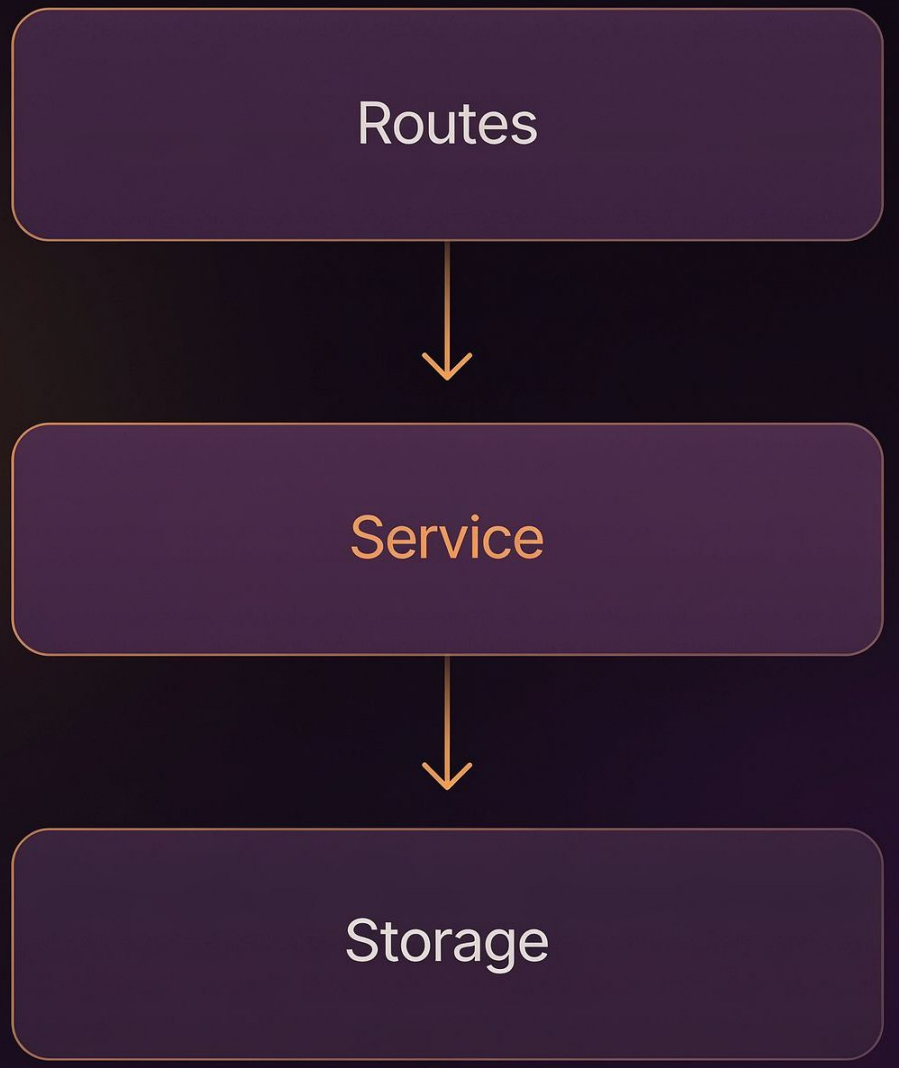
- Status codes at the top
- Business errors and broadcasts in the middle
- null and false at the bottom

Repository interface: `getAll` / `getById` / `create` / `update` / `remove`

Reading, Not Typing

Do not have students type 80 lines. Typing is not the learning activity — reading is.

Prediction device before revealing `taskService.js` v2: ask students to write down what they think disappears from the service. Reveal: no `find`, no `filter`, no `push`, no id generation, no timestamps.





THE HEART OF THE DAY — CONTINUED

Reading the Refactor

Read It Together

- Ids are strings at birth; every comparison is String(...) === String(...), which is why existing numeric ids still match. Same discipline the frontend adopted on Day 4.
- ensureFile, the corruption guard, the atomic .tmp-then-rename — untouched, demoted to private helpers. Day 3's work survives.
- saveTasks is gone. "Replace everything" is no longer something storage can say.
- Ask the room: "Where did the id generation go, and why is that better?" Storage owns identity now — which is precisely the seam Mongo is about to slot into.

Verify

1. Day 3 curl suite, verbatim — including the 400 and the 404
2. Day 4 two-tab live sync — public/ untouched
3. node scripts/stress.js — still loses tasks

> Same engine, better shape. The race is exactly where it was. So what did we buy? The ability to change one variable at a time. Reshape and swap in one move, and when something breaks you have two suspects. Refactor, verify, then swap — this is how you change engines on a running system.

READ → PASTE → LOOK

The Mongo Repository + Seed

> The word mongoose appears in exactly one folder: storage/. If a route or a service imports it, the layers are broken.

1. toTask Preserves the Contract

Mongo says _id and stores an ObjectId; the app has always said id, a string. The repository translates at the boundary — which is exactly why routes, service, and the Day 4 frontend won't notice. The mapping happens once, at the edge, and nowhere else.

2. Invalid IDs Behave as Not-Found

`findById("banana")` throws a CastError; `isValidObjectId` turns nonsense into null, which the service already maps to a 404. No new error paths are introduced.

3. Schema Validation Is the Backstop, Not the Front Door

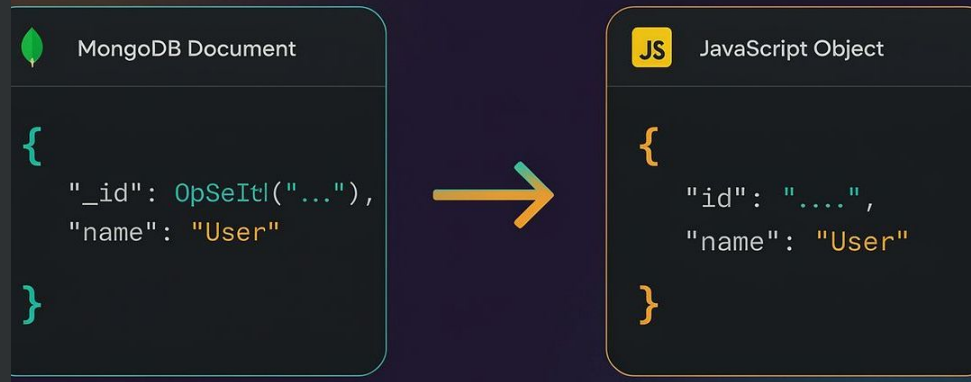
The service throws "Title is required" first — the friendly 400. `required: true` on the schema is defence in depth. The service is the front door; the schema is the lock on the back window.

4. { timestamps: true }

Replaces the hand-rolled `createdAt` and adds `updatedAt` for free. Two lines of schema config retire a whole category of manual bookkeeping.

In Compass, compare the raw document (`_id`, `updatedAt`, `__v`) against what `toTask` exposes. That gap is the mapping. That gap is what a repository is.

The seed script's import list on screen has no Express, no routes, no HTTP anything. It is a third caller, after REST and Socket.IO. On Day 3 the claim was made that the service layer works from HTTP, CLI, sockets, or tests. The seed script is the receipt.



THE PAYOFF

★ The Swap + Verification Ritual

The swap:

```
// Before
const storage = require("../storage/fileStorage");

// After
const storage = require("../storage/mongoStorage");
```

That's it. That's the swap.

Then delete the three `Number(...)` casts from `routes/taskRoutes.js` — the Day 3 deck predicted this exact deletion, by name, in advance. Restart.

The Verification Ritual

In this order, out loud. It builds.

1. **Curl Suite** — Run the full Day 3 curl suite, including the 400 and `/tasks/banana` → 404, not 500. The contract held.
2. **Two Tabs** — Add a task, toggle it, delete it — in two browser tabs simultaneously. Realtime never noticed.
3. **Stress Test × 3** — Run `node scripts/stress.js` three times. Result: 10 / 10, unique 10. The race is dead.
4. **Stop, Restart, Refresh** — Kill the server, restart it, refresh the browser. Data outlived the process.
5. **Delete data/tasks.json Entirely. Refresh.** — Delete the file that held the app together all week. Watch nothing happen. This is the mic drop.

> In the Day 2 monolith this was open-heart surgery on every handler. Here it's two files and a handful of lines, while the live board kept working. Good architecture isn't decoration. You were told that on Day 3. You've now measured it.

The scoreboard: two new files, one require, three deleted casts. `sockets/` untouched. `taskEvents.js` untouched. `public/` — the entire frontend — untouched. Have students run `git diff --stat` and read their own numbers. Real data beats any table.





CODE-ALONG

DX: nodemon & the Watcher Trap

Building the Trap

1. Install nodemon as a dev dependency: npm i -D nodemon
2. Add the scripts to package.json (see right column)
3. Run npm run dev — with no nodemon.json configured yet
4. Click around the task board. Watch the server restart on every single request. Let the room enjoy it.
5. Trace the loop: a request arrives → the logger writes to server.log → nodemon sees a file change → restart. Day 3's logger is the unwitting trigger.

The Fix

1. Add nodemon.json with an ignore list covering logs/, data/, and public/
2. Verify: code edits restart the server, clicks do not, public/ edits do not

The Line Worth Landing

> Every tool you add is a new thing that can fight the tools you already have. Nodemon isn't broken — it's watching exactly what you told it to.

Scripts to Add

```
{
  "scripts": {
    "start": "node server-express.js",
    "dev": "nodemon server-express.js",
    "dev:debug": "nodemon --inspect server-express.js",
    "seed": "node scripts/seed.js",
    "stress": "node scripts/stress.js"
  }
}
```

Note dev:debug — it is used in the debugger segment that follows. Add it now.

nodemon.json

```
{
  "ignore": ["logs/", "data/", "public/"],
  "ext": "js,json"
}
```

DEMO → HANDS-ON

★ The Debugger as an X-ray

The Ladder

`console.log` → `console.error` → `util.inspect` → `node --inspect` → VS Code debugger. Each step is a different kind of question you can ask the running program.

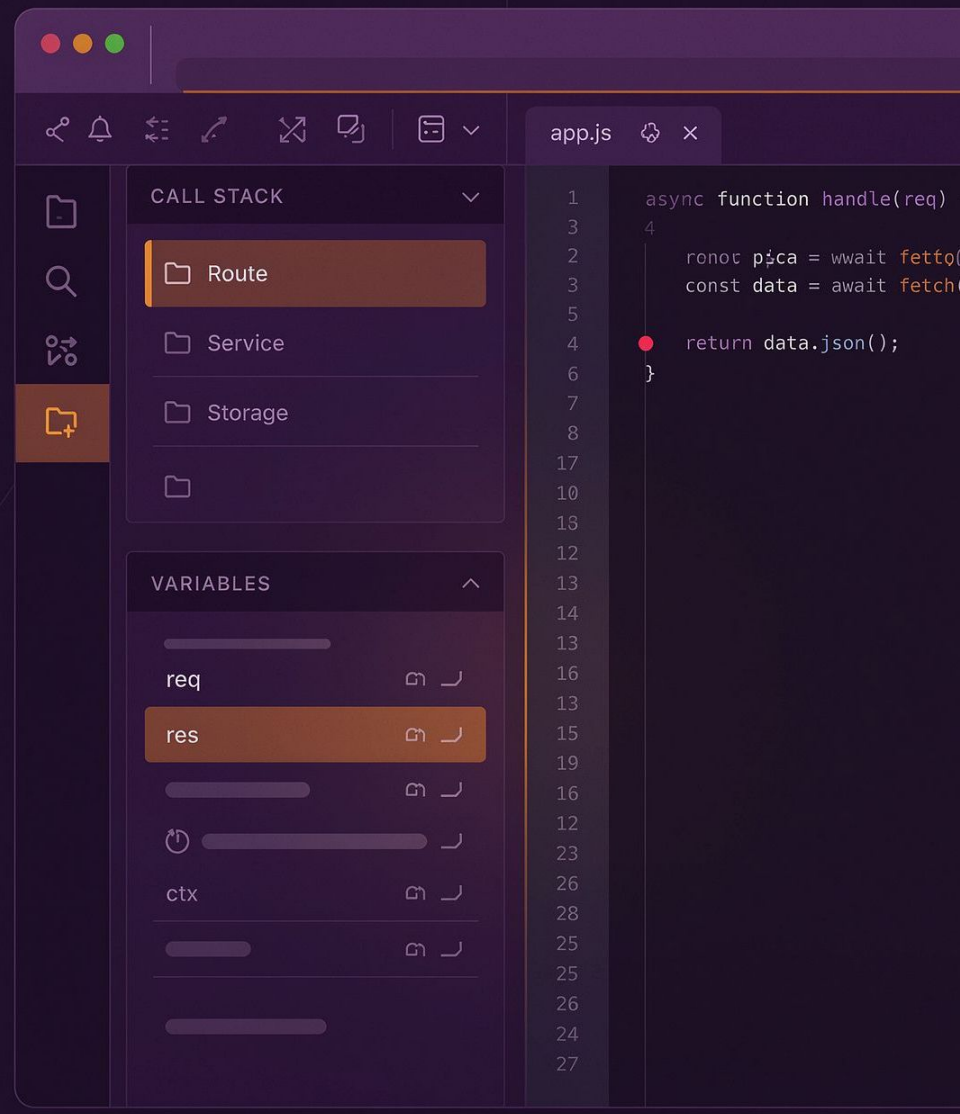
> console.log is a hypothesis you have to guess in advance. A breakpoint is a question you get to ask after you arrive.

Mention `node --inspect` / `--inspect-brk` and `chrome://inspect` in one sentence, then go to VS Code. Paste `.vscode/launch.json` with two deliberate changes from the deck's version: `skipFiles: ["<node_internals>/**"]` added (stops F11 diving into Node internals), and `runtimeExecutable: nodemon` removed (VS Code needs nodemon resolvable on PATH; a devDependency install often isn't, and you get a cryptic spawn error on twenty laptops at once). Plain `node` launches every time. The nodemon variant is `npm run dev:debug` — mention it, don't wire it into `launch.json` today.

You Demo — Once

Breakpoint on the `Task.create(...)` line inside `create` in `storage/mongoStorage.js`. Send a POST. Server pauses. Walk each step as a separate point:

1. Variables panel — `title` is there, exactly as the service passed it.
2. F10 over the `create` call — `doc` appears. Expand it. Not a plain object: `_id` is an ObjectId, plus `$__`, `_doc`, `__v`. "This is the thing your API must never return. This is why `toTask` exists — you're looking at the reason, not reading about it."
3. F10 over `toTask` — now a plain object, `id` a string. The mapping, happening.
4. Call Stack panel — read it bottom to top: Route handler. Service. Storage. "That's not a diagram I drew on Tuesday — that's the actual runtime, and every frame only calls downward. Your architecture isn't a description of this codebase. It's a fact about it, and the debugger just measured it."
5. F5 to resume, confirm the 201.



HANDS-ON LAB

Students Drive

1. **Launch the Debugger** — Press F5 to launch the server under the VS Code debugger (not from the terminal).
2. **Set a Breakpoint** — Set a breakpoint on the `Task.create(...)` line in `storage/mongoStorage.js` — the same line from the demo.
3. **POST a Task** — Send a POST request. Find `title` in the Variables panel. It is there, exactly as the service passed it.
4. **Expand the Raw Document** — F10 past the `create` call. Expand `doc` and find `_id`. It is an `ObjectId` — not a string, not a plain object.
5. **Watch toTask Map It** — F10 past `toTask`. Confirm `id` is now a string. The mapping, happening in front of you.
6. **Screenshot the Call Stack** — Read the Call Stack panel bottom to top: Storage → Service → Route. Screenshot it. That is your proof of the architecture.
7. **Move the Breakpoint** — Move the breakpoint into `update`, then toggle a checkbox from the browser. Same three frames. The layers don't care which transport called them.

> Step 7 is the one that lands: the stack is identical whether the request arrived over REST or over a socket. The layers don't care which transport called them — which is Day 4's lesson, proven by Day 5's tool.

Conditional Breakpoints & Async

Right-click the gutter → Edit Breakpoint → `title.length > 10`. Short title passes, long one stops. Essential for bugs that only appear with specific inputs — and for breakpoints inside loops.

The async warning: a route using `.then()` without `try/catch` means a rejection goes uncaught. `async/await` with `try/catch` and `next(err)` is what lets the centralised error handler work.



CODE-ALONG

/health + Event Loop Lag

The Health Endpoint

- `dbState` from `mongoose.connection.readyState` — The endpoint reports on its dependencies, not just itself. A server that is up but database-less should not claim to be healthy.
- `countDocuments()` — a query that runs in the database. With the file storage, this meant loading everything into Node. That line was impossible two hours ago.
- `uptime` and `heapUsed` — the two numbers that tell you about a slow leak before a user does.

> A server that crashes is obvious. A server that slowly degrades is invisible. By the time anyone notices, it has been building for hours. This endpoint is how a load balancer decides whether to send you traffic — and how you find out first.

Event Loop Lag — Demonstrated

Paste the lag detector. Then break it on purpose with a temporary route:

```
// TEMPORARY — delete after the demo
app.get("/block", (req, res) => {
  const end = Date.now() + 3000;
  while (Date.now() < end) {}     // synchronous. nothing else runs.
  res.json({ blocked: "3s" });
});
```

1. Hit `/block`, then immediately hit `/tasks` in another tab.
2. Everything stops. The lag warning fires.
3. Have students watch the task board during the block, not just the terminal — the live UI freezing is more visceral than a console warning.
4. Delete the route in front of them.

> Day 2, you hit /slow three times in parallel and it finished in three seconds instead of nine — proof that async work doesn't block. This is the same experiment inverted. One synchronous loop, and every connected client — every socket, every request — waits. The event loop isn't a feature you get for free. It's a discipline you have to keep.

The Fix Menu

CPU-heavy work off the main thread: `worker_threads`, a child process, or a job queue. `JSON.parse` on a huge synchronous read is the same bug wearing a suit. One line each — recognise the shape, don't implement it today.



CODE-ALONG

Hardening: Errors & Shutdown

Centralised Error Handler

> Look at your route handlers right now. `err.message === 'Task not found' ? 404 : 500`. You're matching on the text of an error message — change one word in the service and every 404 becomes a 500, silently. Day 3's challenge told you to fix this. Here's the fix.

Walk the mappings:

Error Type	HTTP Status
ValidationError	400 — with the schema's own messages
CastError	400
Duplicate key 11000	409
Everything else	500

- Stack traces in development, never in production
- Routes shrink to: `catch (err) { next(err); }`

Prove it live: `curl -X PUT` with an empty title. Before: a 500. Now: a 400 carrying the schema's message.

Graceful Shutdown

1. `httpServer.close()` — stop accepting new connections
2. `io.close()` — close socket connections
3. `mongoose.disconnect()` — close the database connection
4. `process.exit(0)` — clean exit

Plus a 10-second force-exit timeout.

> SIGINT is you pressing Ctrl+C. SIGTERM is a platform telling you it's over — a deploy, a scale-down, a crash recovery. If you only handle SIGINT, your graceful shutdown works exactly nowhere that matters. Handle both.

> `close()` waits for in-flight requests. If one hangs, you hang forever, and the platform SIGKILLs you anyway. Ten seconds, then take the exit.

Demo it: two browser tabs open, Ctrl+C, and watch the shutdown log its way down the stack — HTTP closed, sockets closed, database disconnected, exit.

What Remains

The remaining items for self-study: helmet, cors, rate limiting, pino. One line each — read what they do before adding them. Rate limiting has a full implementation in the deck — it is the first homework item. Then the deployment paragraph: push the repo, the platform runs `npm start`, set the connection string in their dashboard — the same variable, a different `.env`-equivalent, which is the entire point.



PROTECTED CLOSE

The Whole Journey

- Day 1 — process.argv + fs → a CLI that remembers
- Day 2 — the event loop + http → a server that never blocks
- Day 3 — layers + Express → an API that can grow
- Day 4 — EventEmitter + Socket.IO → an app that pushes
- Day 5 — Mongo + the repository → a system that survives restarts, races, and its own success

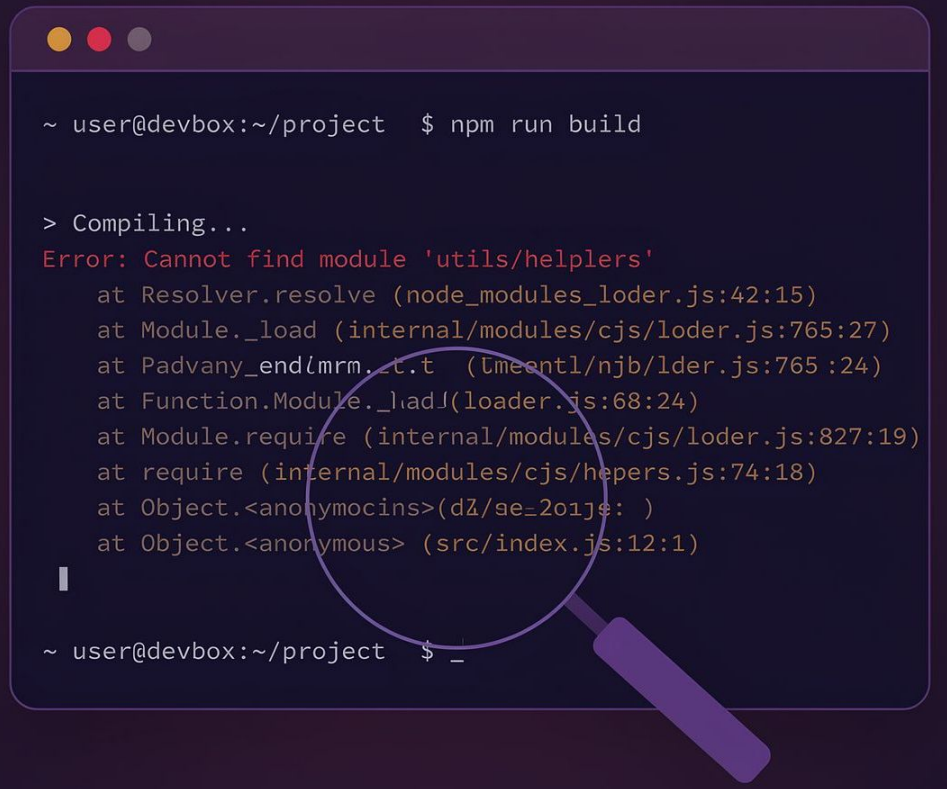
> Same task manager, five days running. Every day's work is still in the final build — process.env from Day 1's runtime, the non-blocking discipline of Day 2, Day 3's layers carrying Day 4's events on top of Day 5's database. Nothing was thrown away. That's what building on fundamentals means.

What today added: real persistence, a debugger workflow that replaces console.log, a health endpoint, graceful shutdown, centralised errors.

The measure of good architecture is not how well it handles what you planned for. It's how easily it handles what you didn't. You have a codebase worth extending. Extend it. Ship something.

Live Failure Playbook

Symptom	30-second fix	If not fixed in 90s
Atlas ETIMEDOUT	Network Access → allowlist current IP	Docker one-liner / class cluster
Authentication failed	Password wrong or special chars not URL-encoded	Docker one-liner
Connection URI undefined	dotenv.config() isn't line 1 / file isn't .env / wrong directory	Buddy up
Data lands in db called "test"	No database name before the ?	—
Requests hang ~10s	Queries before connection — await connectDB() above listen	Solution repo tag
500 on GET /tasks/:id	A `Number()` cast survived, or `isValidObjectId` missing	Solution repo tag
Frontend breaks on task.id	Something returned a raw Mongoose doc — skipped `toTask`	—
Server restarts on every request	nodemon watching `logs/` — nodemon.json missing	Kill nodemon, use npm start
Breakpoint is a hollow grey circle	Source not loaded — check program path in launch.json	Attach config instead
Debugger runs but never pauses	Server started from terminal, not Run & Debug — MOST COMMON	Kill terminal process, F5
Port 9229 already in use	An earlier `--inspect` process is still alive	Kill it, or use different port
Cryptic spawn error on F5	`runtimeExecutable: nodemon` — remove it	Plain node config
F11 dives into Node internals	`skipFiles: ["<node_internals>/**"]` missing	—
Debugger and nodemon fighting	Both running; nodemon restarts detach the debugger	Stop npm run dev before F5





What Goes Home

Practice

- **Queries Challenge** — Extend getAll(options) in both the file and Mongo implementations to support ?done= and ?limit= query parameters. Swap the require back to file storage. Watch both implementations honour the same contract. The interface is real, not aspirational.
- **Rate Limiting** — Self-contained middleware, no new concepts, good practice with Map and 429 status codes. The deck has a full implementation. Unless it was taught in the room, this is the first homework item to reach for.
- **scripts/migrate.js** — Carry the old tasks.json into MongoDB through the repository — approximately 10 lines. Re-proves the layer: the migration script is a fourth caller, after REST, Socket.IO, and the seed script.
- **Contract Test with node:test** — Write one test suite that both fileStorage and mongoStorage must pass. If both pass, the interface is real. If one fails, the interface was aspirational.

Polish

- **AppError Class** — Retires the last of the string-matching error handling, finishing what Day 3's challenge started. A typed error class that carries a status code — routes throw it, the centralised handler catches it.
- **helmet, cors, pino** — One line each. Read what they do before adding them. helmet sets security headers. cors controls cross-origin access. pino replaces console.log with structured, production-grade logging.
- **§10 Debugging Table** — Read once tonight. It covers the full debugging workflow in a reference format. Saves an hour the next time something breaks in production.
- **One Row from Where to Go Next** — Pick one thing from the next-steps table and start it tonight, while the codebase is still warm and the week's context is still in memory. Every row plugs into a layer that already exists.

> Every row plugs into a layer you already have — that's the point of having layers. Don't pick five. Pick the one line your next project needs, and start it tonight while the codebase is still warm.